# Contents

**Distance Queries for Exact PH Offset NURBS** **2**

**1. Objective and scope** **2**

**2. Required interpretation** **3**
2.1 Two distinct distances . . . 3
2.2 Exact-reference model . . . 3
2.3 This is not generic rational-NURBS arc length . . . 3

**3. Public behavior** **4**
3.1 `length` . . . 4
3.2 `arc_length(u)` . . . 4
3.3 `parameter_at_length(s)` . . . 4
3.4 `point_at_length(s)` . . . 4
3.5 Scalar validation and exceptions . . . 5
3.6 Open and closed topology . . . 5
3.7 `cusps` . . . 5

**4. Mathematical kernel** **6**
4.1 Local PH representation . . . 6
4.2 Offset hodograph and rational PH speed . . . 6
4.3 Elementary primitive . . . 7
4.4 Developer-ready Bernstein coefficient formulas . . . 7
4.5 Explicit continuous arctangent increment . . . 10
4.6 Fully expanded closed-form offset distance . . . 12
4.7 Global distance formula . . . 17
4.8 Direct inverse equations . . . 17
4.9 Normative scalar evaluation pseudocode . . . 18
4.10 Worked cubic-PH example and unit-test oracle . . . 22

**5. Monotonicity, cusps, and inverse uniqueness** **23**
5.1 Cusp roots . . . 23
5.2 Strict monotonicity theorem for supported sources . . . 23
5.3 Conditioning at a cusp . . . 23

**6. Structurally required metric certificate** **24**
6.1 Atomic capture . . . 24
6.2 Metric cells . . . 24
6.3 Global prefix index . . . 24
6.4 No lazy unverified metric . . . 24

**7. Certified construction procedure** **25**
7.1 Scale before forming products . . . 25
7.2 Authoritative coefficients . . . 25
7.3 Complete real-root isolation . . . 25
7.4 Certified sign classification . . . 26
7.5 Continuous phase construction . . . 26
7.6 Cell lengths . . . 26
7.7 Inverse seed data . . . 27
7.8 Publication checks . . . 27

**8. Cancellation-resistant elementary evaluation** **27**
8.1 Fast path . . . 27
8.2 Angle evaluation requirements . . . 28

8.3 Catastrophic-cancellation fallback . . . 28
8.4 Range protection . . . 28

**9. Global arc-length evaluation** **29**

**10. Guarded inverse** **29**
10.1 Target reduction . . . 29
10.2 Initial estimate . . . 29
10.3 Safeguarded correction . . . 29
10.4 Termination and best representable parameter . . . 30
10.5 Postcondition . . . 30

**11. Accuracy requirements** **30**
11.1 Forward queries . . . 31
11.2 Inverse queries . . . 31
11.3 Point queries . . . 31
11.4 Reproducibility . . . 31

**12. Reliability and failure rules** **31**
12.1 No false success . . . 31
12.2 Required hard bounds . . . 31
12.3 Unavoidable latency tradeoff . . . 32

**13. Performance requirements** **32**
13.1 Portable complexity contract . . . 32
13.2 Repository Python/Windows profile — isolated special case . . . 32

**14. Verification plan** **33**
14.1 Independent oracle . . . 33
14.2 API and compatibility matrix . . . 33
14.3 Mandatory geometry cases . . . 33
14.4 Required identities and properties . . . 33
14.5 Inverse sampling . . . 34
14.6 Performance and allocation tests . . . 34

**15. Forbidden implementation shortcuts** **35**

**16. Conformance checklist** **35**

**17. Primary references** **35**

# Distance Queries for Exact PH Offset NURBS

**Status:** normative implementation specification and addendum **Applies to:** every `NURBSHandle` returned by `offset(distance)` from every supported open or closed cubic-PH or PH-B-spline source **Does not specify:** implementation source code, generic NURBS arc length, offset trimming, closest-point distance, or a new spline-construction method

The words **SHALL**, **SHALL NOT**, **SHOULD**, and **MAY** are normative. An implementation conforms only if every SHALL requirement is met. Formulas define the required result. They do not require a particular internal data layout or numerical library.

## 1. Objective and scope

Every exact offset NURBS produced by this package SHALL support the common distance-query API already shared by the polynomial PH families, plus the cusp-inspection property specified in Section 3.7:

```
class NURBSHandle:
    @property
    def length(self) -> float: ...

    def arc_length(self, u: Real) -> float: ...

    def parameter_at_length(self, s: Real) -> float: ...

    def point_at_length(self, s: Real) -> NDArray[np.float64]: ...

    @property
    def cusps(self) -> tuple[OffsetCusp, ...]: ...
```

The existing `point(u)` method and all existing inspection properties remain unchanged. These four distance methods and the `cusps` inspection property are the complete public scope of this addendum. `OffsetCusp` is the public tuple-compatible record type `(parameter: float, multiplicity: int)`. In particular, this addendum does not add editing, derivatives, frames, `CurveLocation`, versioning, batch methods, or recursive `offset()` operations to `NURBSHandle`.

The methods SHALL measure distance along the offset locus in source traversal order. They SHALL retain cusps, reversals, loops, and self-intersections. They SHALL NOT trim the locus or replace traversal distance by chord length, closest-point distance, signed displacement, or distance on the source curve.

# 2. Required interpretation

## 2.1 Two distinct distances

This specification uses:

- $d$ for the signed geometric offset passed to `source.offset(d)`; and
- $s$ for nonnegative distance travelled along the resulting offset NURBS.

Positive $d$ remains the package left-normal offset. A negative $d$ remains the right-normal offset. The sign of $d$ does not make $s$ signed.

## 2.2 Exact-reference model

The authoritative mathematical input is the committed binary floating-point source state and the accepted binary floating-point value $d$, each interpreted as an exact real value. The exact-reference offset is

$$z_d(u) = z(u) + dN_L(u), \qquad 0 \le u \le 1.$$

All accuracy statements in this document compare with that exact-reference curve, not with the unknown ideal curve that preceded source construction. This rule makes results testable and independent of algebraically equivalent but differently rounded reconstructions.

## 2.3 This is not generic rational-NURBS arc length

A generic rational NURBS does not have an elementary arc-length function. The required capability exists because this handle is an exact parallel of a regular polynomial PH spline and its generating PH data are known during offset construction. A conforming implementation SHALL preserve a verified metric certificate derived from that data. It SHALL NOT infer PH structure later from rounded public NURBS controls, and it SHALL NOT expose these methods on arbitrary user-constructed NURBS objects unless the same certificate is independently established.

# 3. Public behavior

## 3.1 `length`

`handle.length` SHALL be the total unsigned traversal length

$$L_d = \int_0^1 \|z_d'(u)\| \, du.$$

It SHALL be a finite, strictly positive Python `float`, computed during verified handle construction and returned in $O(1)$ time. For `d == 0`, the exact-reference length and distance functions are those of the captured source. Stored source metric results MAY be reused only when they meet the accuracy contract in Section 11; otherwise they SHALL be recomputed from the captured PH certificate. The rational `point_at_length` path remains subject to the point-evaluation rule in Section 3.4.

## 3.2 `arc_length(u)`

`arc_length(u)` SHALL return

$$A_d(u) = \int_0^u \|z_d'(v)\| \, dv.$$

It SHALL satisfy these endpoint identities exactly in the public arithmetic:

```
arc_length(0.0) == 0.0
arc_length(1.0) == length
```

At a stored source join or exactly representable metric-cell boundary, it SHALL return the canonical stored prefix rather than recompute the same value from an incident cell. A nonrepresentable algebraic cusp remains an exact internal boundary represented by a certified isolating interval; a binary query parameter is compared with it by certified refinement. Exact-reference $A_d$ is strictly increasing. Rounded public results are required only to be nondecreasing because two distinct exact lengths can round to the same floating-point number.

## 3.3 `parameter_at_length(s)`

`parameter_at_length(s)` SHALL return the unique parameter $u \in [0, 1]$ that inverts exact-reference $A_d$, rounded according to Section 10. It SHALL obey:

```
parameter_at_length(0.0) == 0.0
parameter_at_length(length) == 1.0
```

The solve SHALL use the elementary offset-distance function. It SHALL NOT use geometric search, sampled polyline length, numerical quadrature, or a generic unbracketed nonlinear solver.

A public scalar distance can have less resolution than an internal extended prefix. Therefore `parameter_at_length(arc_lengt` cannot recover every $u$ when distinct exact lengths round to the same public float. The inverse is defined by the exact real value of its accepted floating input, not by an arbitrary preimage of a prior rounded forward result. This limitation SHALL NOT cause internal cells or prefixes to be merged.

## 3.4 `point_at_length(s)`

`point_at_length(s)` SHALL be semantically identical to:

```
handle.point(handle.parameter_at_length(s))
```

It MAY share the metric lookup and NURBS span lookup to avoid repeated work. It SHALL use the existing verified rational evaluation path and SHALL return the same value as the two-call expression for the same accepted argument, subject only to a documented one-ULP coordinate difference caused by a fused internal path. Endpoint results SHALL use the existing exact endpoint path.

### 3.5 Scalar validation and exceptions

The methods SHALL use the existing package scalar rules.

- Accept Python and NumPy real scalars.
- Reject Booleans, arrays, sequences, complex values, and non-real objects with `TypeError`.
- `u` is valid on $[0, 1]$. NaN and material excursions raise `ParameterOutOfRangeError`. Values within the existing parameter endpoint slack clamp to that endpoint. In the repository binary64 profile, this slack is four times machine epsilon.
- `s` is valid on $[0, L_d]$. NaN and material excursions raise `ArcLengthOutOfRangeError`. Values no more than four ULPs of $L_d$ outside an endpoint clamp to that endpoint.
- Infinities are never clamped.
- A safeguarded inverse that cannot establish its acceptance certificate raises `ArcLengthInversionError`.
- A runtime elementary evaluation whose accuracy cannot be certified within the resource bound raises `NumericalPrecisionError`.
- Failure to build and verify the metric certificate is an atomic offset construction failure and raises `OffsetConstructionError`. A handle with partially initialized distance methods SHALL NOT be published.

The existing structured fields SHALL identify the operation, source span or metric cell, offending quantity, measured residual, required bound, and iteration or precision level when applicable.

### 3.6 Open and closed topology

The distance domain is always the one traversal interval $[0, L_d]$. `parameter_at_length` does not wrap. On a closed handle, $s = 0$ returns $u = 0$ and $s = L_d$ returns $u = 1$, although `point(0)` and `point(1)` are the same geometric point. Self-intersections introduce no special cases.

### 3.7 `cusps`

`handle.cusps` SHALL return the certified offset cusps of the handle as an immutable ascending tuple of `OffsetCusp(parameter, multiplicity)` records. `OffsetCusp` SHALL be a named, tuple-compatible record type with the fields `parameter` (Python `float`) and `multiplicity` (Python `int`).

The exact-reference cusp set is the set of zero-speed parameters of the exact-reference offset: the distinct real roots of $G_d$ on every source span (Section 5.1), including roots at source-span endpoints and internal joins. The returned records SHALL satisfy all of the following.

- One record per distinct representable stationary parameter, in strictly increasing `parameter` order.
- Every `parameter` SHALL be a binary64 global parameter within two ulps of its exact-reference root (one ulp of certified local bracket plus the exactly-analyzed local-to-global affine rounding), obtained from the certified isolating bracket of Section 7.3. When the root owns a stored metric-cell boundary, the record SHALL equal that boundary, so `arc_length(parameter)` returns the stored cusp prefix of Section 3.2.
- Every `multiplicity` SHALL be the certified root multiplicity of Section 7.3. An odd value marks an offset hodograph direction reversal; an even value marks a tangential zero-speed contact without a reversal (Section 5.1).
- Two distinct exact roots whose refined parameters coalesce to the same or adjacent representable values MAY be reported as one record carrying the larger multiplicity. This is the representable-boundary limit of Section 6.2 and SHALL be documented.
- For `d == 0`, for spans with $\tau \equiv 0$, and for every cusp-free offset, the tuple SHALL be empty.

The tuple SHALL be assembled during verified metric construction from the complete certified root isolation of Section 7.3 - never from sampling, later re-derivation, or rounded public NURBS controls - and returned in $O(1)$ time with no construction work on the query. It SHALL be preserved by copying and by serialization through the rebuilt certificate (Section 6.1), and it SHALL fail with the same typed error as the distance methods on any handle that carries no verified metric certificate.

A cusp record is an inspection value. It creates no trimming, segmentation, or region operation; those remain outside `NURBSHandle`.

# 4. Mathematical kernel

## 4.1 Local PH representation

On one regular source polynomial span, use a positively oriented local parameter $t \in [0, 1]$ and write the source hodograph in user units as

$$z'(t) = \lambda w(t)^2,$$

where $w(t) = a(t) + ib(t)$ is a complex polynomial of degree $m$ and $\lambda > 0$ is constant on the span. The constant includes the source's spatial normalization and any positive local parameter-width factor. Define

$$\sigma(t) = |w(t)|^2 = a(t)^2 + b(t)^2 > 0,$$

$$\tau(t) = 2\,\mathrm{Im}(\overline{w(t)}w'(t)) = 2(ab' - ba').$$

Then

$$v(t) = \|z'(t)\| = \lambda\sigma(t),$$

$$\theta'(t) = \frac{\tau(t)}{\sigma(t)}, \qquad \kappa(t) = \frac{\tau(t)}{\lambda\sigma(t)^2},$$

where $\theta$ is a continuous lift of the source tangent angle. These expressions are invariant under the sign ambiguity $w \mapsto -w$.

For the repository's normalized PH-B-spline span, $\lambda = H_x h_i$, where $H_x$ is the spatial normalization scale and $h_i$ is the local source parameter width. Equivalently, calculations can use $\hat{d} = d/H_x$ and the normalized polynomial $h_i\sigma^2 - \hat{d}\tau$. For a normalized cubic span, the corresponding width factor is one. This paragraph is a repository mapping only; the rest of the specification does not depend on it.

## 4.2 Offset hodograph and rational PH speed

Let

$$T = \frac{w^2}{\sigma}, \qquad N_L = iT.$$

Since $N_L' = -\theta' T$, differentiation gives

$$\boxed{z_d'(t) = \left(\lambda\sigma(t) - d\frac{\tau(t)}{\sigma(t)}\right) T(t).}$$

Define the signed offset-speed numerator and signed speed

$$G_d(t) = \lambda\sigma(t)^2 - d\tau(t),$$

$$q_d(t) = \frac{G_d(t)}{\sigma(t)}.$$

The geometric speed is

$$\boxed{v_d(t) = |q_d(t)| = \frac{|G_d(t)|}{\sigma(t)}.}$$

Thus the offset is rational PH. The signed rational function $q_d$ is a PH speed representative; the unsigned geometric speed is piecewise rational because its sign changes at offset cusps.

The degrees satisfy

$$\deg \sigma \le 2m, \qquad \deg \tau \le 2m-2, \qquad \deg G_d \le 4m.$$

The apparent degree $2m-1$ leading term of $\mathrm{Im}(\bar{w}w')$ vanishes because it is real. Here the zero polynomial has degree $-\infty$; in particular, a degree-zero straight span has $\tau \equiv 0$ and $G_d = \lambda\sigma^2 > 0$ for every finite $d$.

## 4.3 Elementary primitive

Let

$$S(t) = \lambda \int_0^t \sigma(x)\,dx$$

be the polynomial source arc length on this span, and let

$$\Theta(t) = 2\,\mathrm{Arg}_c\, w(t)$$

be a continuous, unwrapped tangent-angle lift. Since $\Theta' = \tau/\sigma$,

$$\boxed{R_d(t) = S(t) - d\Theta(t)}$$

is an elementary primitive of the signed offset speed: $R_d' = q_d$. On any interval $[a,b]$ that contains no sign change of $G_d$, let $\eta \in \{-1,+1\}$ be its sign. The exact unsigned offset length is

$$\boxed{D_d(a,t) = \eta\left([S(t)-S(a)] - d[\Theta(t)-\Theta(a)]\right), \quad a \le t \le b.}$$

This is the normative elementary distance formula. The arctangent term is not optional: rational PH curves generally have rational speed but need not have a rational arc-length function.

## 4.4 Developer-ready Bernstein coefficient formulas

This section removes all coefficient derivation from the implementation task. The formulas are normative. An implementation MAY use an algebraically equivalent basis, but its results SHALL be checked against these identities.

#### 4.4.1 Input preimage and derivative

Let the local complex preimage be given in the degree-$m$ Bernstein basis:

$$w(t) = \sum_{j=0}^{m} w_j B_j^m(t), \qquad w_j = A_j + iB_j.$$

Here $A_j$ and $B_j$ are real coefficient components; $B_j^m(t)$ is the Bernstein basis function. The shared letter does not denote the same object.

For $m \geq 1$, its derivative has degree $m-1$ Bernstein controls

$$\boxed{e_j = m(w_{j+1} - w_j), \qquad 0 \leq j \leq m-1,}$$

so that

$$w'(t) = \sum_{j=0}^{m-1} e_j B_j^{m-1}(t).$$

For $m = 0$, set $w'(t) = 0$, $\tau(t) = 0$, and use the straight-span rules stated below.

Throughout Sections 4.4–4.8, a binomial coefficient with an invalid lower index is zero. The displayed summation bounds avoid such terms explicitly.

#### 4.4.2 Speed polynomial `sigma`

Write

$$\sigma(t) = |w(t)|^2 = \sum_{k=0}^{2m} \rho_k B_k^{2m}(t).$$

The required real Bernstein coefficients are

$$\boxed{\rho_k = \frac{1}{\binom{2m}{k}} \sum_{i=\max(0,k-m)}^{\min(m,k)} \binom{m}{i}\binom{m}{k-i} \operatorname{Re}(w_i \overline{w_{k-i}}), \quad 0 \leq k \leq 2m.}$$

In real components,

$$\operatorname{Re}(w_i \overline{w_j}) = A_i A_j + B_i B_j.$$

The sum is real in exact arithmetic. A nonzero computed imaginary remainder is a numerical error indicator, not part of the coefficient.

#### 4.4.3 Turning numerator `tau`

For $m \geq 1$, represent $\tau$ initially at degree $n_\tau = 2m - 1$:

$$\tau(t) = 2\operatorname{Im}(\overline{w(t)} w'(t)) = \sum_{k=0}^{n_\tau} \xi_k B_k^{n_\tau}(t).$$

Its Bernstein coefficients are

$$\boxed{\xi_k = \frac{2}{\binom{2m-1}{k}} \sum_{i=\max(0,k-m+1)}^{\min(m,k)} \binom{m}{i}\binom{m-1}{k-i} \operatorname{Im}(\overline{w_i} e_{k-i}), \quad 0 \le k \le 2m-1.}$$

In real components, if $e_j = E_j + iF_j$,

$$\operatorname{Im}(\overline{w_i} e_j) = A_i F_j - B_i E_j.$$

Although the true power degree of $\tau$ is at most $2m-2$, the degree-$(2m-1)$ Bernstein representation above is valid and avoids a numerical degree-reduction decision. Exact cancellation of the leading power coefficient SHALL be verified independently.

#### 4.4.4 Squared-speed and cusp polynomial `G`

First form

$$\sigma(t)^2 = \sum_{k=0}^{4m} p_k B_k^{4m}(t)$$

with

$$\boxed{p_k = \frac{1}{\binom{4m}{k}} \sum_{i=\max(0,k-2m)}^{\min(2m,k)} \binom{2m}{i}\binom{2m}{k-i} \rho_i \rho_{k-i}, \quad 0 \le k \le 4m.}$$

For $m \ge 1$, elevate the degree-$(2m-1)$ coefficients $\xi_j$ to degree $4m$. Let $r = 4m - (2m-1) = 2m+1$. Then

$$\tau(t) = \sum_{k=0}^{4m} \tilde{\xi}_k B_k^{4m}(t),$$

where

$$\boxed{\tilde{\xi}_k = \frac{1}{\binom{4m}{k}} \sum_{j=\max(0,k-r)}^{\min(2m-1,k)} \binom{2m-1}{j}\binom{r}{k-j} \xi_j, \quad 0 \le k \le 4m.}$$

The signed offset-speed numerator is therefore

$$G_d(t) = \sum_{k=0}^{4m} g_k B_k^{4m}(t),$$

with the explicit controls

$$\boxed{g_k = \lambda p_k - d\tilde{\xi}_k.}$$

For $m = 0$, the special case is

$$\boxed{\sigma(t) = \rho_0 = |w_0|^2, \quad \tau(t) = 0, \quad G_d(t) = \lambda \rho_0^2 > 0.}$$

No root isolation is needed on such a straight span.

### 4.4.5 Point evaluation of the metric polynomials

At any $t \in [0,1]$, evaluate the preceding Bernstein polynomials by de Casteljau or a proved equivalent bounded-error evaluator:

$$\sigma(t) = \mathrm{deCasteljau}(\rho, t),$$

$$\tau(t) = \mathrm{deCasteljau}(\xi, t),$$

$$G_d(t) = \mathrm{deCasteljau}(g, t).$$

The signed and unsigned offset speeds used by the inverse are then exactly

$$\boxed{q_d(t) = \frac{G_d(t)}{\sigma(t)}, \qquad v_d(t) = \frac{|G_d(t)|}{\sigma(t)}.}$$

## 4.5 Explicit continuous arctangent increment

This section gives the closed-form angle term that was implicit in Section 4.3.

### 4.5.1 Dot and determinant arguments

For two parameters $a \le t$ in one source span, let

$$w(a) = A_a + iB_a, \qquad w(t) = A_t + iB_t.$$

Define

$$\boxed{X(a,t) = \mathrm{Re}\big(w(t)\overline{w(a)}\big) = A_t A_a + B_t B_a,}$$

$$\boxed{Y(a,t) = \mathrm{Im}\big(w(t)\overline{w(a)}\big) = B_t A_a - A_t B_a.}$$

Thus $X$ is the dot product of the two preimage vectors and $Y$ is their oriented determinant. Source regularity guarantees that $X$ and $Y$ are not both zero.

Let

$$\alpha(a,t) = \mathrm{atan2}(Y(a,t), X(a,t)) \in (-\pi, \pi].$$

For platform independence, `atan2` in every formula means the exact quadrant-aware function

$$\boxed{\mathrm{atan2}(y,x) = \begin{cases} \arctan(y/x), & x > 0, \\ \arctan(y/x) + \pi, & x < 0,\ y \ge 0, \\ \arctan(y/x) - \pi, & x < 0,\ y < 0, \\ +\pi/2, & x = 0,\ y > 0, \\ -\pi/2, & x = 0,\ y < 0, \end{cases}}$$

where arctan has range $(-\pi/2, \pi/2)$. The case $x = y = 0$ cannot occur because $w$ is nonzero. At $x < 0, y = 0$, the exact principal value is $+\pi$; the incident phase rule in Section 4.5.2 handles continuous passage through that cut without relying on the sign of floating-point zero.

The principal value $\alpha$ is not by itself the continuous change in argument. Let $\omega(a,t) \in \mathbb{Z}$ be the certified winding correction defined in Section 4.5.2. The continuous preimage-angle change is

$$\boxed{\Delta\phi(a,t) = \alpha(a,t) + 2\pi\omega(a,t).}$$

Because the unit tangent is $T = (w/|w|)^2$, its continuous angle change is

$$\boxed{\Delta\Theta(a,t) = 2\Delta\phi(a,t) = 2\left[\operatorname{atan2}(Y(a,t), X(a,t)) + 2\pi\omega(a,t)\right].}$$

This is the explicit arctangent expression required by the elementary distance formula.

### 4.5.2 Exact winding-correction rule

The integer $\omega$ SHALL be determined topologically, not inferred by rounding an angle quotient.

Choose the fixed nonzero reference $c = w(0) = A_0 + iB_0$ and define

$$x_c(t) = \operatorname{Re}(w(t)\overline{c}) = A_t A_0 + B_t B_0,$$

$$y_c(t) = \operatorname{Im}(w(t)\overline{c}) = B_t A_0 - A_t B_0.$$

No product conversion is needed to isolate the phase-cut crossings. In the same degree-$m$ Bernstein basis as $w$, the real controls are directly

$$\boxed{x_{c,j} = \operatorname{Re}(w_j\overline{c}) = A_j A_0 + B_j B_0,}$$

$$\boxed{y_{c,j} = \operatorname{Im}(w_j\overline{c}) = B_j A_0 - A_j B_0, \qquad 0 \le j \le m.}$$

Thus

$$x_c(t) = \sum_{j=0}^{m} x_{c,j} B_j^m(t), \qquad y_c(t) = \sum_{j=0}^{m} y_{c,j} B_j^m(t).$$

Isolate every root of $y_c$ for which $x_c < 0$. These are the crossings of the principal-argument cut, the ray opposite to $c$. Define an integer $k_c(t)$, initially $k_c(0) = 0$, and process the cut roots in increasing parameter order:

- if $y_c$ changes from positive to negative, increment $k_c$ by one;
- if $y_c$ changes from negative to positive, decrement $k_c$ by one; and
- if $y_c$ has even multiplicity and does not change sign, leave $k_c$ unchanged.

All signs and multiplicities in this rule SHALL be certified. The integer is constant on each open interval between cut roots.

At every cut root $r$, store two incident phase pairs:

$$(\beta_-(r), k_-(r)) = \lim_{t \uparrow r} (\beta(t), k_c(t)),$$

$$(\beta_+(r), k_+(r)) = \lim_{t \downarrow r} (\beta(t), k_c(t)).$$

The principal components at the cut are the signed one-sided values $+\pi$ or $-\pi$, not the result of passing a rounded signed zero to the host `atan2`. The update rule above guarantees

$$\boxed{\beta_-(r) + 2\pi k_-(r) = \beta_+(r) + 2\pi k_+(r).}$$

For an oriented interval $[a, t]$, use the right-incident pair at its start $a$ and the left-incident pair at its end $t$. If an endpoint is not a cut root, its unique interval pair is used. This convention makes an angle increment continuous and additive when a query endpoint equals a cut root.

Away from an exact cut root, define

$$\beta(t) = \operatorname{atan2}(y_c(t), x_c(t)) \in (-\pi, \pi],$$

$$\boxed{\phi_c(t) = \beta(t) + 2\pi k_c(t).}$$

Then $\phi_c(0) = 0$ and $\phi_c$ is the continuous argument change of $w$ from 0 to $t$. Consequently,

$$\boxed{\Delta\phi(a, t) = \phi_c(t) - \phi_c(a).}$$

For cancellation-resistant evaluation with the relative angle $\alpha$, set

$$\delta_0 = \beta(t) - \beta(a),$$

and define the integer branch adjustment

$$h(a, t) = \begin{cases} +1, & \delta_0 > \pi, \\ -1, & \delta_0 \le -\pi, \\ 0, & -\pi < \delta_0 \le \pi. \end{cases}$$

Then the winding correction used in Section 4.5.1 is

$$\boxed{\omega(a, t) = k_c(t) - k_c(a) + h(a, t).}$$

At an exact cut-root endpoint, $\beta(a), k_c(a)$ or $\beta(t), k_c(t)$ in these formulas mean the incident pair just specified. At the exact values $\delta_0 = \pm\pi$, use the stated $(-\pi, \pi]$ convention for the relative angle $\alpha$. Comparisons with $\pi$ SHALL use certified enclosures. An implementation SHALL NOT compute $\omega$ as a floating-point rounding of $(\Delta\phi - \alpha)/(2\pi)$.

## 4.6 Fully expanded closed-form offset distance

### 4.6.1 Algebraic power-basis form

For clarity, suppose

$$w(t) = \sum_{j=0}^{m} c_j t^j.$$

Define the real power coefficients of $\sigma = |w|^2$ by

$$\boxed{s_n = \operatorname{Re} \sum_{j=\max(0,n-m)}^{\min(m,n)} c_j \overline{c_{n-j}}, \quad 0 \le n \le 2m.}$$

Then

$$S(t) - S(a) = \lambda \sum_{n=0}^{2m} \frac{s_n}{n+1} \left(t^{n+1} - a^{n+1}\right).$$

Let $[a, b]$ be one metric cell and let $\eta = \operatorname{sign} G_d$ on its interior. Substituting the explicit arctangent increment from Section 4.5 gives

$$\boxed{\begin{aligned} D_d(a,t) = \eta \Bigg[ & \lambda \sum_{n=0}^{2m} \frac{s_n}{n+1} \left(t^{n+1} - a^{n+1}\right) \\ & - 2d \left(\operatorname{atan2}(Y(a,t), X(a,t)) + 2\pi\omega(a,t)\right) \Bigg], \qquad a \le t \le b. \end{aligned}}$$

This is a complete closed-form distance expression. It contains only finite polynomial sums, one `atan2`, the constant $\pi$, one certified integer, and ordinary arithmetic. No integration remains for the developer to derive.

With the `atan2` arguments substituted, the same formula is

$$\boxed{\begin{aligned} D_d(a,t) = \eta \Bigg[ & \lambda \sum_{n=0}^{2m} \frac{s_n}{n+1} \left(t^{n+1} - a^{n+1}\right) \\ & - 2d \left(\operatorname{atan2}\left(B_t A_a - A_t B_a,\ A_t A_a + B_t B_a\right) + 2\pi\omega(a,t)\right) \Bigg]. \end{aligned}}$$

This fully substituted boxed form is the shortest standalone formula a developer can translate directly.

Its derivative can be checked without geometric reasoning:

$$\frac{d}{dt} \operatorname{atan2}(Y(a,t), X(a,t)) = \frac{\operatorname{Im}(\overline{w(t)} w'(t))}{|w(t)|^2} = \frac{\tau(t)}{2\sigma(t)}.$$

Therefore, away from a cusp,

$$\frac{d}{dt} D_d(a,t) = \eta \left(\lambda\sigma(t) - d\frac{\tau(t)}{\sigma(t)}\right) = \frac{|G_d(t)|}{\sigma(t)}.$$

The power-basis expression is normative as an identity. Direct evaluation of $t^{n+1} - a^{n+1}$ is not the preferred floating-point algorithm near $a$. Use Section 4.6.2 for the production path.

#### 4.6.2 Stable Bernstein form

Let

$$\sigma(x) = \sum_{j=0}^{2m} \rho_j^{[a,t]} B_j^{2m}\Big(\frac{x-a}{t-a}\Big), \qquad a \le x \le t,$$

where $\rho_j^{[a,t]}$ are the Bernstein controls obtained by exact-parameter de Casteljau restriction of the original $\rho_j$ to $[a,t]$. The integral of every degree-$2m$ Bernstein basis function over its unit interval is $1/(2m+1)$. Therefore

$$\boxed{\Delta S(a,t) = S(t) - S(a) = \lambda \frac{t-a}{2m+1} \sum_{j=0}^{2m} \rho_j^{[a,t]}.}$$

The stable coefficient-authority forward distance is thus

$$\boxed{D_d(a,t) = \eta \Bigg[ \lambda \frac{t-a}{2m+1} \sum_{j=0}^{2m} \rho_j^{[a,t]} - 2d\left(\operatorname{atan2}(Y(a,t), X(a,t)) + 2\pi\omega(a,t)\right) \Bigg].}$$

This boxed expression is the normative coefficient-authority formula for a forward in-cell query.

The interval restriction is fully specified by the following basis-neutral pseudocode, where `split(c, x)` is one de Casteljau split and returns control arrays for the left and right subintervals:

```
restrict(c, a, b):
    require 0 <= a < b <= 1
    if a == 0:
        return split(c, b).left
    right = split(c, a).right
    local_b = (b - a) / (1 - a)
    return split(right, local_b).left
```

The divisions and interval endpoints in this operation SHALL carry the error bounds required by Section 8. Subdivision changes the parameterization of the restricted controls to $[0,1]$; the explicit factor $t-a$ restores the source parameter measure.

Production code SHALL normally avoid repeating this restriction for every query. Restrict $\sigma$ once during metric-cell construction to the whole cell $[a,b]$:

$$\sigma(a + (b-a)x) = \sum_{j=0}^{n} r_j B_j^n(x), \qquad n = 2m, \quad 0 \le x \le 1.$$

Compile the forward antiderivative controls

$$\boxed{f_0 = 0, \qquad f_{j+1} = f_j + \frac{r_j}{n+1}, \quad 0 \le j \le n,}$$

and define

$$Q_f(x) = \sum_{j=0}^{n+1} f_j B_j^{n+1}(x).$$

For

$$x = \frac{t-a}{b-a},$$

the source-length increment is exactly

$$\boxed{\Delta S(a,t) = \lambda(b-a)Q_f(x).}$$

For reverse evaluation, compile

$$\boxed{r_0^{\mathrm{rev}} = 0, \qquad r_{j+1}^{\mathrm{rev}} = r_j^{\mathrm{rev}} + \frac{r_{n-j}}{n+1}, \quad 0 \le j \le n,}$$

and define

$$Q_r(y) = \sum_{j=0}^{n+1} r_j^{\mathrm{rev}} B_j^{n+1}(y), \qquad y = \frac{b-t}{b-a}.$$

Then

$$\boxed{\Delta S(t,b) = \lambda(b-a)Q_r(y).}$$

The production forward distance can therefore be evaluated with fixed cell data as

$$\boxed{D_d(a,t) = \eta\left\{\lambda(b-a)Q_f\left(\frac{t-a}{b-a}\right) - 2d\left[\operatorname{atan2}(Y(a,t), X(a,t)) + 2\pi\omega(a,t)\right]\right\}.}$$

The forward and reverse antiderivative controls SHALL be stored with the cell. A compiled compensated-Horner form MAY provide the ordinary $O(m)$ path. The Bernstein controls above remain the stable fallback and coefficient authority. A fast Horner value is accepted only under its Section 8 error bound.

### 4.6.3 Cell total

For a whole metric cell $[a,b]$, its exact length is obtained without any new formula:

$$\boxed{\ell_{[a,b]} = \eta\left[\lambda\frac{b-a}{2m+1}\sum_{j=0}^{2m}\rho_j^{[a,b]} - 2d\left(\operatorname{atan2}(Y(a,b), X(a,b)) + 2\pi\omega(a,b)\right)\right] > 0.}$$

This value is the cell-prefix increment stored by Section 6.

#### 4.6.4 Reverse remainder

For stable evaluation near the right endpoint $b$, define

$$X_r(t,b) = \mathrm{Re}(w(b)\overline{w(t)}),$$

$$Y_r(t,b) = \mathrm{Im}(w(b)\overline{w(t)}),$$

and use the winding correction $\omega(t,b)$. The exact remaining distance from $t$ to $b$ is

$$\boxed{D_d^{\mathrm{rev}}(t,b) = \eta\left[\lambda\frac{b-t}{2m+1}\sum_{j=0}^{2m}\rho_j^{[t,b]} - 2d\left(\mathrm{atan2}(Y_r(t,b),X_r(t,b)) + 2\pi\omega(t,b)\right)\right].}$$

Using the precompiled reverse antiderivative of Section 4.6.2, the equivalent production formula is

$$\boxed{D_d^{\mathrm{rev}}(t,b) = \eta\left\{\lambda(b-a)Q_r\left(\frac{b-t}{b-a}\right) - 2d\left[\mathrm{atan2}(Y_r(t,b),X_r(t,b)) + 2\pi\omega(t,b)\right]\right\}.}$$

It obeys

$$D_d(a,t) + D_d^{\mathrm{rev}}(t,b) = \ell_{[a,b]}.$$

The production evaluator SHALL compute the nearer of the forward distance and reverse remainder directly. It SHALL NOT obtain a small reverse remainder by subtracting $D_d(a,t)$ from the cell total.

#### 4.6.5 Repository normalized-coordinate specialization

This subsection is the isolated package-specific specialization. It is not part of the platform-independent mathematical contract.

For a repository PH-B-spline span, let $H_x > 0$ be the spatial normalization scale, let $h_i > 0$ be the stored local parameter-width factor, and set

$$\hat{d} = \frac{d}{H_x}.$$

The normalized cusp polynomial, physical offset speed, and physical forward distance are

$$\boxed{\widehat{G_d}(t) = h_i\sigma(t)^2 - \hat{d}\tau(t),}$$

$$\boxed{v_d(t) = H_x\frac{|\widehat{G_d}(t)|}{\sigma(t)},}$$

$$\boxed{D_d(a,t) = H_x\eta\left[h_i(b-a)Q_f\left(\frac{t-a}{b-a}\right) - 2\hat{d}\left(\operatorname{atan2}(Y(a,t),X(a,t)) + 2\pi\omega(a,t)\right)\right].}$$

The reverse formula replaces $Q_f((t-a)/(b-a))$ by $Q_r((b-t)/(b-a))$ and uses $Y_r, X_r, \omega(t,b)$. For a repository cubic span, use the same expressions with $h_i = 1$. An implementation SHALL either:

1. use these normalized formulas and multiply the final distance and speed by $H_x$; or
2. use the general formulas with $\lambda = H_x h_i$ and physical $d$.

It SHALL NOT mix physical $d$ with $\widehat{G_d}$ or omit the final $H_x$. The quotient $d/H_x$ follows the extended-precision rule in Section 7.1.

#### 4.6.6 Exact zero-offset and straight-span shortcuts

If $d = 0$, then

$$G_0 = \lambda\sigma^2 > 0, \qquad D_0(a,t) = \Delta S(a,t).$$

If $\tau \equiv 0$ on a span, the span is straight and the same distance identity holds for every finite $d$:

$$\boxed{\tau \equiv 0 \quad \Longrightarrow \quad D_d(a,t) = \Delta S(a,t).}$$

These cases SHALL skip cusp isolation, winding lookup, `atan2`, and the subtraction of the angle term. This is both exact and faster; evaluating an algebraically zero angle term through floating arithmetic is unnecessary.

### 4.7 Global distance formula

Let source span $i$ occupy global parameter interval $[u_i, u_{i+1}]$ and let

$$t = \frac{u - u_i}{u_{i+1} - u_i}.$$

Suppose $t$ lies in metric cell $j = [a_j, b_j]$ of that source span. Let $P_{i,j}$ be the extended accumulated offset length of all preceding source spans and metric cells. Then the complete global query is

$$\boxed{A_d(u) = P_{i,j} + D_{d,i}(a_j, t).}$$

Near $b_j$, the equivalent cancellation-resistant form is

$$\boxed{A_d(u) = P_{i,j+1} - D^{\mathrm{rev}}_{d,i}(t, b_j).}$$

At $u = 0$ return zero. At $u = 1$ return the stored total. At a representable cell boundary return its stored prefix. These formulas apply without change to open and closed splines; closed topology changes endpoint geometry, not the one-traversal distance sum.

### 4.8 Direct inverse equations

Let a target have already been reduced to one metric cell $[a, b]$.

#### 4.8.1 Forward solve

For a local forward target $s_f \in [0, \ell_{[a,b]}]$, solve

$$\boxed{F_f(t) = D_d(a,t) - s_f = 0.}$$

Inside the cell,

$$\boxed{F_f'(t) = v_d(t) = \frac{|G_d(t)|}{\sigma(t)} = \eta \frac{G_d(t)}{\sigma(t)} > 0.}$$

The explicit Newton proposal is

$$\boxed{t_N = t - \frac{D_d(a,t) - s_f}{|G_d(t)|/\sigma(t)}.}$$

#### 4.8.2 Reverse solve

For a target closer to the right endpoint, let $s_r = \ell_{[a,b]} - s_f$ and solve

$$\boxed{F_r(t) = D_d^{\mathrm{rev}}(t,b) - s_r = 0.}$$

Its derivative is

$$\boxed{F_r'(t) = -v_d(t) = -\frac{|G_d(t)|}{\sigma(t)} < 0,}$$

so the explicit Newton proposal is

$$\boxed{t_N = t + \frac{D_d^{\mathrm{rev}}(t,b) - s_r}{|G_d(t)|/\sigma(t)}.}$$

At a cusp endpoint, $G_d = 0$ and neither Newton quotient is evaluated there. Use the cusp seed of Section 7.7 and the safeguarded bracket procedure of Section 10.

#### 4.8.3 Residual authority

The forward residual SHALL be evaluated by the boxed formula in Section 4.6.2. The reverse residual SHALL be evaluated by the boxed formula in Section 4.6.4. The speed SHALL be evaluated from the boxed formula in Section 4.4.5. A polynomial approximation, LUT interpolation, or NURBS chord length is not an authoritative residual or derivative.

### 4.9 Normative scalar evaluation pseudocode

The following pseudocode fixes the correspondence between the mathematical symbols and an implementation. It omits only the certified arithmetic operations, whose error rules are specified in Sections 7 and 8.

```
compile_span_metric(preimage_controls w[0..m], lambda, offset d):
    if m == 0:
        rho = [abs(w[0])**2]
        tau = [0]
        G = [lambda * rho[0]**2]
```

```
    cusp_roots = []
else:
    e[j] = m * (w[j+1] - w[j])

    for k = 0 .. 2*m:
        rho[k] = 0
        for i = max(0, k-m) .. min(m, k):
            j = k - i
            rho[k] += choose(m,i) * choose(m,j) * real(w[i]*conjugate(w[j]))
        rho[k] /= choose(2*m,k)

    for k = 0 .. 2*m-1:
        tau[k] = 0
        for i = max(0, k-m+1) .. min(m, k):
            j = k - i
            tau[k] += choose(m,i) * choose(m-1,j) * imag(conjugate(w[i])*e[j])
        tau[k] *= 2 / choose(2*m-1,k)

    for k = 0 .. 4*m:
        sigma_squared[k] = 0
        for i = max(0, k-2*m) .. min(2*m, k):
            j = k - i
            sigma_squared[k] += choose(2*m,i) * choose(2*m,j) * rho[i]*rho[j]
        sigma_squared[k] /= choose(4*m,k)

        tau_elevated[k] = 0
        for j = max(0, k-(2*m+1)) .. min(2*m-1, k):
            tau_elevated[k] += (
                choose(2*m-1,j) * choose(2*m+1,k-j) * tau[j]
            )
        tau_elevated[k] /= choose(4*m,k)
        G[k] = lambda * sigma_squared[k] - d * tau_elevated[k]

    if G is the exact zero polynomial:
        fail metric construction as required by Section 5.2
    if d == 0 or tau is the exact zero polynomial:
        cusp_roots = []
    else:
        cusp_roots = certified_all_real_roots(G, [0, 1])

if d == 0 or tau is the exact zero polynomial:
    cut_roots = []
    phase data is unused
else:
    c = w[0]
    cut_x[j] = real(w[j] * conjugate(c))
    cut_y[j] = imag(w[j] * conjugate(c))
    if cut_y is the exact zero polynomial:
        cut_roots = []
        phase is identically zero
    else:
        cut_roots = certified_all_real_roots(cut_y, [0, 1])
        retain only cut roots whose certified cut_x value is negative
        assign oriented cut indices by the rule in Section 4.5.2
        store the left-incident and right-incident (beta, cut_index) pair
```

```
                at every cut root

    boundaries = sorted([0, cusp_roots..., 1])
    for each consecutive cell [a, b]:
        eta = certified_sign(G at any certified interior point)
        r[0..2*m] = restrict(rho, a, b)

        forward[0] = 0
        reverse[0] = 0
        for j = 0 .. 2*m:
            forward[j+1] = forward[j] + r[j] / (2*m + 1)
            reverse[j+1] = reverse[j] + r[2*m-j] / (2*m + 1)

        if d == 0 or tau is the exact zero polynomial:
            cell.length = lambda * (b-a) * forward[2*m+1]
        else:
            cell.length = evaluate the Section 4.6.3 boxed formula
        store [a, b], eta, r, forward, reverse, phase data, length

    build extended forward and reverse prefixes
    perform every publication check in Section 7.8

phase_pair(span, t, side):
    if t is a certified cut root:
        return stored incident pair selected by side
    c = span.preimage_at_zero
    w_t = eval_preimage(span, t)
    beta = atan2(imag(w_t * conjugate(c)), real(w_t * conjugate(c)))
    k = oriented cut index of the open cut interval containing t
    return beta, k

winding_correction(span, a, t):
    # At a cut root, phase_pair returns the stored signed-pi limit.
    beta_a, k_a = phase_pair(span, a, side="right")
    beta_t, k_t = phase_pair(span, t, side="left")

    delta0 = beta_t - beta_a
    if delta0 > pi:
        h = +1
    elif delta0 <= -pi:
        h = -1
    else:
        h = 0
    return k_t - k_a + h

forward_cell_distance(cell, t):
    # cell = [a, b], with sign eta and source-span data
    if t == cell.a:
        return 0
    if t == cell.b:
        return cell.length

    x = (t - cell.a) / (cell.b - cell.a)
    delta_source = cell.span.lambda * (cell.b - cell.a) * (
        certified_eval(cell.forward_source_antiderivative, x)
```

```
    )
    if cell.offset == 0 or cell.span.turning_is_identically_zero:
        return delta_source

    w_a = eval_preimage(cell.span, cell.a)
    w_t = eval_preimage(cell.span, t)

    X = real(w_t * conjugate(w_a))
    Y = imag(w_t * conjugate(w_a))
    omega = winding_correction(cell.span, cell.a, t)
    delta_theta = 2 * (atan2(Y, X) + 2*pi*omega)

    return cell.eta * (delta_source - cell.offset * delta_theta)

reverse_cell_distance(cell, t):
    if t == cell.b:
        return 0
    if t == cell.a:
        return cell.length

    y = (cell.b - t) / (cell.b - cell.a)
    delta_source = cell.span.lambda * (cell.b - cell.a) * (
        certified_eval(cell.reverse_source_antiderivative, y)
    )
    if cell.offset == 0 or cell.span.turning_is_identically_zero:
        return delta_source

    w_t = eval_preimage(cell.span, t)
    w_b = eval_preimage(cell.span, cell.b)

    X = real(w_b * conjugate(w_t))
    Y = imag(w_b * conjugate(w_t))
    omega = winding_correction(cell.span, t, cell.b)
    delta_theta = 2 * (atan2(Y, X) + 2*pi*omega)

    return cell.eta * (delta_source - cell.offset * delta_theta)

offset_arc_length(u):
    if u == 0: return 0
    if u == 1: return total_length

    span, t = locate_source_span(u)
    cell = locate_metric_cell(span, t)
    if t == cell.a: return cell.prefix
    if t == cell.b: return cell.next_prefix

    if t - cell.a <= cell.b - t:
        return cell.prefix + forward_cell_distance(cell, t)
    return cell.next_prefix - reverse_cell_distance(cell, t)

offset_parameter_at_length(s):
    if s == 0: return 0
    if s == total_length: return 1

    cell = locate_extended_prefix(s)
```

```
s_f = exact_subtract(s, cell.prefix)
s_r = exact_subtract(cell.next_prefix, s)

if s_f <= s_r:
    solve forward_cell_distance(cell, t) - s_f == 0
    derivative = abs(eval_G(cell.span, t)) / eval_sigma(cell.span, t)
else:
    solve reverse_cell_distance(cell, t) - s_r == 0
    derivative = -abs(eval_G(cell.span, t)) / eval_sigma(cell.span, t)

use cusp seed or LUT seed
apply the bracket and acceptance rules of Section 10
map accepted local t to the public global parameter u
verify the public residual before return
```

Every multiplication by $d$, every determinant, every polynomial sum, every angle, and every final subtraction in this pseudocode is subject to the certified fast/fallback rules of Sections 7 and 8. Replacing those operations by raw binary64 expressions makes the pseudocode nonconforming.

### 4.10 Worked cubic-PH example and unit-test oracle

Take the linear preimage

$$w(t) = 1 + it, \qquad 0 \le t \le 1, \qquad \lambda = 1.$$

Then

$$\sigma(t) = 1 + t^2, \qquad \tau(t) = 2, \qquad G_d(t) = (1 + t^2)^2 - 2d,$$

and

$$S(t) = t + \frac{t^3}{3}.$$

For $0 \le a \le t \le 1$,

$$w(t)\overline{w(a)} = (1 + at) + i(t - a),$$

so

$$X(a, t) = 1 + at > 0, \qquad Y(a, t) = t - a, \qquad \omega(a, t) = 0.$$

The complete in-cell distance reduces to

$$\boxed{D_d(a, t) = \eta \left[ (t - a) + \frac{t^3 - a^3}{3} - 2d \operatorname{atan2}(t - a, 1 + at) \right].}$$

This also verifies the tangent-angle identity

$$\operatorname{atan2}(t - a, 1 + at) = \arctan(t) - \arctan(a).$$

For $d = 1$, the unique interior cusp is

$$c = \sqrt{\sqrt{2} - 1} = 0.6435942529055827\ldots .$$

The sign is $\eta = -1$ on $[0, c]$ and $\eta = +1$ on $[c, 1]$. The two exact cell-length expressions and their reference values are

$$\ell_0 = 2\arctan(c) - c - \frac{c^3}{3} = 0.41126166475721382\ldots ,$$

$$\ell_1 = \left(1 + \frac{1}{3} - c - \frac{c^3}{3}\right) - 2\left(\frac{\pi}{4} - \arctan(c)\right) = 0.17379867129565052\ldots ,$$

$$\boxed{L_1 = \ell_0 + \ell_1 = 0.58506033605286434\ldots .}$$

A conforming implementation SHALL include this case as a deterministic unit test for coefficient construction, cusp isolation, sign partitioning, `atan2` orientation, cell prefixes, forward/reverse evaluation, and inverse queries on both sides of the cusp.

# 5. Monotonicity, cusps, and inverse uniqueness

## 5.1 Cusp roots

The real roots of $G_d$ are exactly the zero-speed parameters of the offset on a regular source span. They are geometric stationary points, commonly called offset cusps in this package. A root of odd multiplicity reverses the offset hodograph direction. A root of even multiplicity touches zero without a direction reversal. Both cases SHALL remain in the curve and in its distance function.

## 5.2 Strict monotonicity theorem for supported sources

For every valid handle, $A_d(u)$ is strictly increasing, even when its derivative is zero at isolated parameters.

Proof. For $d = 0$, $q_d = \lambda\sigma > 0$. For $d \neq 0$, suppose $q_d$ vanished on a nonempty open interval. Then $1 - d\kappa = 0$ there, so the regular source span would have constant nonzero curvature $1/d$ and would trace a circular arc. A nonconstant polynomial map cannot trace a circle on an interval: the polynomial identity $|z - c|^2 = R^2$ would have a nonzero positive leading coefficient unless $z$ were constant. That contradicts source regularity. Therefore $G_d$ is not identically zero. Its zeros are finite, $v_d$ is positive on an open subset of every nonzero parameter interval, and the integral of $v_d$ over that interval is positive. QED.

Consequences:

- the inverse is single-valued;
- cusp parameters do not require an arbitrary left/right inverse policy;
- no zero-length parameter plateau may be silently introduced by numerical rounding; and
- a metric compiler that concludes $G_d \equiv 0$ has found corrupted source data or a numerical defect and SHALL fail construction.

## 5.3 Conditioning at a cusp

If $G_d$ has multiplicity $r \geq 1$ at $c$, then locally

$$v_d(t) = C|t - c|^r + O(|t - c|^{r+1}),$$

$$A_d(t) - A_d(c) = \frac{C}{r+1}\,\mathrm{sign}(t - c)|t - c|^{r+1} + O(|t - c|^{r+2}).$$

The inverse parameter is intrinsically ill-conditioned at the cusp. No implementation can promise small relative parameter error there. Acceptance SHALL instead use an arc-length residual and the best-representable-parameter rule in Section 10. The resulting geometric position remains well conditioned by traversal distance.

# 6. Structurally required metric certificate

## 6.1 Atomic capture

Offset construction SHALL atomically capture enough immutable source data to evaluate and verify Sections 4 and 5 after the source is edited or destroyed. The data MAY be stored as preimage coefficients, derived speed and turning coefficients, or another algebraically equivalent certificate. It SHALL not be a live back-reference.

The handle's private construction proof SHALL cover both:

1. the existing rational NURBS geometry certificate; and
2. the distance certificate specified here.

Deserialization SHALL rebuild derived caches and reverify all structural invariants before making the handle usable. Copying, pickling, threads, and later source edits SHALL not change distance results.

## 6.2 Metric cells

Each source polynomial span SHALL be partitioned into ordered metric cells. A metric cell has:

- a global parameter interval and its local affine map;
- one source-span identity;
- a certified constant sign $\eta$ of $G_d$ in its open interior;
- endpoint cusp multiplicities, if any;
- restricted speed controls $r_j$ and the explicit forward/reverse antiderivative controls $f_j$, $r_j^{\mathrm{rev}}$ of Section 4.6.2;
- data for a continuous phase evaluation of $\Theta$;
- a positive cell length with a certified error enclosure;
- a compensated or extended prefix length;
- optional inverse seed data that cannot be used for acceptance; and
- error bounds for every fast evaluator.

Metric cells need not coincide with the rational Bezier leaves introduced by positive-weight refinement. `num_spans` continues to mean public rational NURBS spans. The metric-cell count remains private.

## 6.3 Global prefix index

Cell lengths SHALL be accumulated in traversal order with compensated, double-double, expansion, or stronger arithmetic. A plain left-to-right binary64 sum is nonconforming. Store both forward and reverse information so that a query near either global endpoint does not subtract nearly equal total lengths.

The exact-reference prefixes are strictly increasing. Internal prefix representations SHALL preserve that order even when adjacent public floats would tie. Public `float` conversion happens only at the API boundary.

## 6.4 No lazy unverified metric

The metric certificate SHALL be completed before handle publication. A mutable lazy cache would weaken immutability, thread safety, serialization, and failure atomicity. Read-only derived micro-caches MAY be created lazily only if their absence cannot affect correctness and publication already verified the complete fallback path.

# 7. Certified construction procedure

## 7.1 Scale before forming products

Calculations SHALL use the source normalized frame or an equivalent power-of-two scaled frame. Forming $\lambda\sigma^2$, $d\tau$, or $d\Theta$ directly in user units is forbidden when an intermediate can overflow or underflow although the final result is representable.

Coefficient construction SHALL use exponent tracking and error-free or error-bounded products and sums. A recommended normalized form for the repository is

$$\widehat{G}_d = h_i\sigma^2 - \hat{d}\tau, \qquad \hat{d} = d/H_x,$$

with the division represented in extended precision when one rounded binary64 quotient would change a root classification. Multiplication of $\widehat{G}_d$ by any certified positive scale is allowed because it does not change roots or signs.

The determinant $ab' - ba'$ and the subtraction in $G_d$ are cancellation sites. A raw pair of rounded products followed by subtraction SHALL NOT be an acceptance value. Use fused multiply-add, error-free `two_product` plus `two_sum`, floating-point expansions, interval arithmetic, or stronger arithmetic.

## 7.2 Authoritative coefficients

Build $\sigma$, $\tau$, and $G_d$ by the explicit Bernstein product and degree-elevation formulas of Section 4.4 from the captured preimage. Binomial weights SHALL be generated exactly as integers or rationals before the final arithmetic conversion. Independently verify:

$$\sigma = |w|^2, \qquad \tau = 2\,\mathrm{Im}(\bar{w}w'), \qquad G_d = \lambda\sigma^2 - d\tau.$$

Each coefficient SHALL carry an enclosure that contains its exact-reference value. If a sign or root count depends on overlapping coefficient intervals, increase precision; do not guess a sign or snap a coefficient to zero.

## 7.3 Complete real-root isolation

All distinct real roots of $G_d$ in $[0, 1]$ SHALL be isolated and their multiplicities SHALL be known or safely bounded. This includes:

- simple roots;
- even-multiplicity tangencies;
- roots at either source-span endpoint;
- clusters of roots separated by only a few floating-point numbers; and
- degree drops caused by exact coefficient cancellation.

`numpy.roots`, companion-matrix eigenvalues, sampled sign scans, and an uncertified tolerance-based deduplication MAY supply seeds only. None may establish completeness or multiplicity.

A conforming portable strategy is one of:

1. interpret the finite input coefficients as exact rationals, use a subresultant square-free decomposition and a Sturm or Bernstein-Descartes isolator, then refine each interval with safeguarded interval Newton; or
2. use adaptive directed-rounding interval or ball arithmetic with a proved complete real-root count and increase precision until every interval is classified.

The isolator SHALL return disjoint ordered intervals, each containing one distinct root, plus proof that the complement contains no root. Root intervals SHALL be refined until their uncertainty contributes no more than one quarter of the cell-length error budget. Because $q_d$ vanishes at a root, a simple-root location error

contributes only second order to the length; this fact MAY be used in the bound but SHALL NOT be assumed for an unclassified root.

At a source join, endpoint roots from incident spans are owned by the right-hand span, except $u = 1$, which is owned by the last span. Incident one-sided speed laws remain separate if curvature is discontinuous. No positive-width cell may be lost during deduplication.

## 7.4 Certified sign classification

Evaluate $G_d$ with a certified enclosure at one interior point of each root-complement interval. The enclosure SHALL exclude zero. Its sign is $\eta$. Multiplicity parity and incident signs SHALL agree. Any disagreement is a construction failure, not a reason to discard a root.

## 7.5 Continuous phase construction

Calling `atan2(b(t), a(t))` independently at two parameters is insufficient: the preimage can cross an arctangent branch cut or wind around the origin.

For each source span, choose $c = w(0) \neq 0$ and define

$$x_c(t) = \mathrm{Re}(\bar{c}w(t)), \qquad y_c(t) = \mathrm{Im}(\bar{c}w(t)).$$

Then $x_c(0) > 0$ and $y_c(0) = 0$. Isolate every real root of $y_c$ on $[0, 1]$ with the same certified machinery used for $G_d$. At a root, $x_c$ cannot be zero because source regularity gives $w \neq 0$.

Only crossings with $x_c < 0$ cross the principal `atan2` cut. Maintain the integer winding counter $k(t)$ with the oriented update rule in Section 4.5.2, using the certified signs of $y_c$ on adjacent intervals, and define the continuous phase

$$\phi(t) = \mathrm{atan2}(y_c(t), x_c(t)) + 2\pi k(t), \qquad \phi(0) = 0,$$

$$\Theta(t) = 2\phi(t).$$

At a tangential contact with the cut, choose the continuous one-sided limit; do not use the sign of a rounded zero. If $y_c \equiv 0$, regularity makes the preimage phase constant and $\Theta(t) \equiv 0$ on that span.

An alternative half-plane subdivision and phase-lift proof is conforming if it proves the same continuous branch. Principal-angle differences without a branch proof are forbidden.

## 7.6 Cell lengths

For every sign cell $[a, b]$, compute

$$\ell = \eta\{[S(b) - S(a)] - d[\Theta(b) - \Theta(a)]\} > 0.$$

The direct implementation SHALL use the fully expanded cell-total formula in Section 4.6.3, including its `atan2` arguments and winding integer.

The source term SHOULD be evaluated as the integral of the restricted Bernstein speed on $[a, b]$, not by subtracting two large source prefixes. If $\sigma$ restricted to $[a, b]$ has degree $n$ and Bernstein coefficients $c_0, \ldots, c_n$, then

$$S(b) - S(a) = \lambda \frac{b - a}{n + 1} \sum_{j=0}^{n} c_j.$$

Use an accurate sum and the stored regularity bound. Compute the angle term on its certified continuous branch.

The two positive-sized terms in the braces can nearly cancel. Use the cancellation protocol in Section 8. A cell is accepted only when a positive length and its requested rounding are certified. An unresolved zero or negative result SHALL NOT be clamped.

### 7.7 Inverse seed data

The implementation SHOULD store a small monotone lookup table per cell or per source span. Table values SHALL be computed by the authoritative metric evaluator and stored with extended prefixes. Rounded duplicate table values MAY be removed. A table is a seed and bracket accelerator only. It SHALL NOT be an acceptance oracle.

At a cusp endpoint, store the root multiplicity $r$ and a certified leading coefficient for the local law

$$D(c, c+\delta) = C_r|\delta|^{r+1} + O(|\delta|^{r+2}), \qquad C_r > 0.$$

This supplies the well-scaled inverse seed

$$|\delta| \approx (s/C_r)^{1/(r+1)}$$

instead of a linear seed with zero derivative.

### 7.8 Publication checks

Before publication, verify at minimum:

1. source-span and global parameter coverage is exactly $[0, 1]$ with no gap or overlap;
2. all root intervals are ordered, disjoint, and complete;
3. every cell has a proved sign and positive exact-reference length;
4. phase winding is continuous through every branch crossing;
5. compensated prefixes are ordered and the last equals `length`;
6. forward and reverse cell lengths agree within their enclosures;
7. elementary derivative identity $R'_d = G_d/\sigma$ holds by independent coefficient or high-precision checks;
8. deterministic interior values agree with an independent high-precision oracle;
9. inverse seed brackets contain their targets; and
10. geometry and metric certificates refer to the same captured source version and the same exact offset $d$.

## 8. Cancellation-resistant elementary evaluation

### 8.1 Fast path

For a query $t$ in a cell $[a, b]$, evaluate from the nearer endpoint. The forward and reverse exact forms are

$$D_f(t) = \eta\{[S(t) - S(a)] - d[\Theta(t) - \Theta(a)]\},$$

$$D_r(t) = \eta\{[S(b) - S(t)] - d[\Theta(b) - \Theta(t)]\}.$$

Evaluate the source increment from restricted or forward/reverse Bernstein data. Evaluate $w$ with de Casteljau, compensated Horner, or a proved equivalent kernel. Compute the angle with the stored phase branch.

The subtraction SHALL use at least double-double-equivalent precision and shall produce an explicit absolute error bound $E$. A fast value is accepted only if its sign and requested public rounding are determined despite $E$. Merely testing that the rounded result is nonnegative is nonconforming.

For low degrees, a compiled power-basis Horner evaluator MAY be used when its forward-error bound passes. Stable Bernstein evaluation remains the fallback. All polynomial evaluations SHALL use endpoint-special forms at 0 and 1.

## 8.2 Angle evaluation requirements

The `atan2` implementation SHALL have a documented error bound. If the host library does not promise correct rounding, the fast path SHALL include its proved or conservatively measured bound and SHALL fall back whenever that bound can change the result. Exact multiples of $\pi$ SHALL be represented in the same extended arithmetic used by the winding counter; adding a rounded $2\pi k$ repeatedly is forbidden.

Compute small angle differences with

$$\operatorname{atan2}\left(\operatorname{Im}(w_1\overline{w_0}), \operatorname{Re}(w_1\overline{w_0})\right)$$

on a certified branch, rather than subtract two nearly equal principal angles. Compute the determinant and dot product with error-free or error-bounded arithmetic. Use `atan2`, not `atan(y/x)`.

## 8.3 Catastrophic-cancellation fallback

The difference $\Delta S - d\Delta\Theta$ is genuinely ill-conditioned when an offset nearly stalls. Binary64 or double-double arithmetic alone cannot resolve every finite input. A conforming implementation SHALL provide both of these fallbacks:

1. **Local rational series.** Around every cusp endpoint, and optionally around other high-condition points, expand $q_d = G_d/\sigma$. If $G_d = \sum g_j x^j$, $\sigma = \sum c_j x^j$, and $q_d = \sum q_j x^j$, compute

   $$q_0 = g_0/c_0, \qquad q_n = \frac{g_n - \sum_{j=1}^{n} c_j q_{n-j}}{c_0}.$$

   Integrate with

   $$\int_0^x q_d(y)dy = \sum_{n=0}^{N} \frac{q_n}{n+1} x^{n+1} + E_N(x).$$

   Construction SHALL provide a certified radius and remainder bound. The series is accepted only when the remainder and rounding enclosure prove the requested result.
2. **Adaptive elementary evaluation.** Re-evaluate the exact formula with increasing precision, including polynomial products, phase, $\pi$, `atan2`, and the final subtraction, until the result's sign and public rounding are certified.

The local series preserves microsecond-scale service near ordinary simple cusps. Adaptive precision handles adversarial cancellation and is expected to be rare. Both paths SHALL have resource accounting. The package profile SHALL cap adaptive precision at a documented value of at least 4096 bits; reaching the cap raises `NumericalPrecisionError` with no returned value. It SHALL NOT loop indefinitely or silently use a low-accuracy approximation.

## 8.4 Range protection

Use scaled `hypot`-style evaluation for $|w|$, exponent-separated products, and scale-before-square rules. Underflowed nonzero speed, an infinite intermediate with finite final result, and `inf - inf` are implementation defects, not geometric zeros.

If exact-reference total offset length is outside the finite positive public scalar range, including a positive length that would round to zero, `offset(d)` SHALL raise `OffsetConstructionError` during atomic metric construction. Returning a zero, infinite, or NaN `.length`, or a geometry-only handle with failing distance methods, is forbidden.

# 9. Global arc-length evaluation

For an accepted `u`:

1. return exact endpoint values for 0 or 1;
2. locate the source span and metric cell by a binary search over immutable breakpoints;
3. return the stored prefix on an exact canonical boundary;
4. evaluate a forward or reverse in-cell distance as in Section 8;
5. combine it with the nearer compensated global prefix by an error-free sum; and
6. round once to the public scalar after the accuracy certificate passes.

The result SHALL be checked against $[0, L_d]$ using its error enclosure. A slightly negative or over-length raw number may clamp only when the enclosure contains the exact endpoint. Any material violation raises `NumericalPrecisionError`.

No variable-sized buffer or array allocation is permitted on the ordinary scalar fast path, apart from the result array required by `point_at_length`. Managed-runtime scalar objects and mandated public return objects do not count toward this rule. Temporary fixed-degree work areas SHOULD be preallocated, stack allocated, or thread local.

# 10. Guarded inverse

## 10.1 Target reduction

After validation:

1. return exact endpoints immediately;
2. locate the unique metric cell with extended-prefix comparisons;
3. form the local target by error-free subtraction;
4. if it is closer to the cell's right endpoint, solve the reverse remainder problem and recover the forward parameter only after acceptance; and
5. obtain a strict initial bracket from the cell endpoints or verified LUT.

The prefix search SHALL define a deterministic owner when a public target equals a rounded prefix. Compare the exact floating input against extended prefixes. Do not compare only their rounded high parts.

## 10.2 Initial estimate

Use, in priority order:

1. an exact cell endpoint hit;
2. the cusp asymptotic seed when the selected endpoint speed is zero;
3. monotone interpolation in a verified LUT bracket; or
4. the bracket midpoint.

Clamp a seed only to the strict bracket interior. A seed cannot be accepted without an authoritative residual evaluation.

## 10.3 Safeguarded correction

Let $F(t)$ be forward distance or reverse remainder and let $y$ be the local target. Maintain an invariant bracket $[l, h]$ such that $F(l) \le y \le F(h)$ for the increasing formulation. Every authoritative evaluation SHALL update the bracket before proposing the next point.

The primary proposal is Newton:

$$t_{new} = t - \frac{F(t) - y}{v_d(t)}.$$

Evaluate $v_d = |G_d|/\sigma$ with scaling and a certified nonnegative error bound. Reject Newton and use a bracket step if:

- the derivative enclosure contains zero;
- the candidate is nonfinite or not strictly inside the bracket;
- it does not reduce a conservative residual bound; or
- it lies too close to the same bracket endpoint to ensure progress.

A safeguarded inverse-quadratic, secant, or TOMS-748-type bracket step MAY precede bisection. It SHALL preserve the bracket. Arithmetic midpoint

$$l + (h - l)/2$$

SHALL be used instead of $(l + h)/2$ when it is representably interior.

### 10.4 Termination and best representable parameter

Residual is the primary convergence measure. Parameter-step size alone SHALL NOT accept a result, especially at a cusp.

Let $\hat{t}$ be a binary floating-point candidate and let $E_F$ be the radius of a certified exact-reference enclosure for $F(\hat{t})$. The normal fast acceptance gate is

$$|F(\hat{t}) - y| \le E_F + 4\,\mathrm{ulp}(y) + 32\epsilon y,$$

where $E_F$ is the complete elementary-evaluation error bound. The implementation SHOULD use the tighter of the forward and reverse bounds.

If no candidate passes before the ordinary iteration limit, finish with a floating-point ordered search over the **public global parameter** $u$. Searching local-$t$ encodings is conforming only when the local/global affine map is proved to preserve the same adjacent public values. Bisect the ordered global encodings, not only their real-number midpoint, until two adjacent representable global parameters bracket $y$. Map each trial to its local cell coordinate with an error-free or certified affine operation. This takes a bounded number of steps for a fixed format, including subnormal parameters. Return the one whose certified distance is closer to $y$; break an exact tie toward the smaller global parameter. This is the **best-representable-parameter rule** and is an acceptance certificate even when no representable parameter meets the normal residual gate. If the two distance-error enclosures overlap enough to prevent this comparison, increase evaluation precision until they separate or prove an exact tie. Reaching the precision cap is a typed failure, not a guessed choice.

For IEEE 754 binary64, at most 64 ordered-encoding bisections are needed once the bracket is mapped to nonnegative finite encodings. The complete inverse SHALL have a hard bound on ordinary iterations, ordered searches, and adaptive precision. Exhaustion raises `ArcLengthInversionError` or `NumericalPrecisionError`; an unverified iterate is never returned.

### 10.5 Postcondition

After mapping local $t$ to global $u$, clamp only by the proved affine-cell interval. Re-evaluate `arc_length(u)` through the public-authority kernel and verify the normal residual gate or best-representable certificate. This final check prevents a local/global mapping error, wrong cell, stale prefix, or phase-branch defect from escaping.

## 11. Accuracy requirements

Let $A^*(u)$ and $L^*$ denote exact-reference values.

### 11.1 Forward queries

For every finite accepted input, construction and query evaluation SHALL produce a certified enclosure containing the exact-reference value. The returned `.length` and `arc_length(u)` SHALL be faithfully rounded: one of the two consecutive representable values that bracket the exact result. Equivalently, away from overflow and underflow boundaries,

$$|\widehat{A} - A^*| \le \mathrm{ulp}(A^*).$$

For zero or a subnormal exact value, use an absolute bound of one smallest positive subnormal value. Endpoint identities in Section 3 override the general bound and are exact.

### 11.2 Inverse queries

`parameter_at_length` SHALL satisfy either:

1. the certified residual gate in Section 10.4; or
2. the certified best-representable-parameter rule.

At a regular point, the implementation SHALL also report in internal tests the implied estimate

$$|\delta u| \lesssim \frac{|\delta s|}{v_d(u)}.$$

This estimate is diagnostic and does not replace the residual. At cusps, no relative parameter-error promise is made.

### 11.3 Point queries

`point_at_length(s)` inherits the existing `point(u)` homogeneous NURBS forward-error contract. In addition, the traversal-distance error caused by the inverse SHALL meet Section 11.2. Tests SHALL separate inverse error from rational point-evaluation error.

### 11.4 Reproducibility

Results SHALL be deterministic for one source state, $d$, arithmetic profile, and library version. Bitwise equality across different `atan2` libraries is not portable and is not required. Cross-platform results SHALL satisfy the same enclosures, root counts, branch winding, and faithful-rounding bounds.

## 12. Reliability and failure rules

### 12.1 No false success

Accuracy and reliability take priority over a latency target. If an implementation cannot certify a root count, phase branch, cell sign, cell length, elementary value, or inverse residual, it SHALL raise the applicable typed exception. It SHALL NOT return a sampled, clamped, last-iterate, NaN, infinite, or tolerance-guessed result.

### 12.2 Required hard bounds

Every subdivision, root refinement, Newton correction, bracket correction, ordered-float search, and precision escalation SHALL have a documented hard resource bound. Hitting a bound is a typed failure with diagnostic fields. No input may cause unbounded iteration.

### 12.3 Unavoidable latency tradeoff

There is no finite fixed precision that resolves every cancellation possible from finite floating-point inputs. Therefore a universal fixed nanosecond/microsecond worst-case bound and faithful rounding are mutually incompatible. Conformance requires a bounded fast path and a certified adaptive path. A performance claim SHALL state whether adaptive cases are included and SHALL report their incidence.

## 13. Performance requirements

### 13.1 Portable complexity contract

Let $C$ be the number of metric cells, $m$ the source preimage degree, $q$ the public rational degree, and $I$ the number of safeguarded inverse evaluations. Ordinary-path complexity SHALL be:

| Operation | Required complexity | Allocation |
|---|---|---|
| `.length` | $O(1)$ | none |
| `.cusps` | $O(1)$ | none beyond the returned records |
| `arc_length` | $O(\log C + m)$ | none |
| `parameter_at_length` | $O(\log C + Im)$ | none |
| `point_at_length` | inverse plus existing $O(q^2)$ de Boor, or a proved faster equivalent | result array only |

The allocation column excludes required scalar return objects and managed-runtime scalar temporaries. It forbids variable-sized work buffers and temporary numerical arrays on the ordinary path, as specified in Section 9.

The common path SHOULD need no more than two seed/Newton evaluations plus one authoritative acceptance evaluation. Root isolation, phase compilation, and adaptive coefficient work belong to handle construction, not ordinary queries.

Construction storage SHALL be $O((N + R)m + P + J)$, where $N$ is the source-span count, $R$ is the cusp-root count, $P$ is the phase-cut record count, and $J$ is the inverse-seed node count. This bound counts arithmetic values; an adaptive exact-arithmetic implementation SHALL also report limb storage. A dense table indexed by all public NURBS controls is not required.

### 13.2 Repository Python/Windows profile — isolated special case

This subsection is not a portable semantic requirement. It applies to the current repository reference environment: 64-bit CPython 3.14 on Windows, NumPy binary64 arrays, and the project's documented Intel i9-13900K benchmark host. Other implementations SHALL replace it with measured targets for their platform.

After cache warm-up, common non-adaptive scalar queries SHOULD remain in these orders of magnitude:

| Query | Cubic offset target | Source PH degree at most 17 target | Configured maximum source PH degree 33 target |
|---|---|---|---|
| `.length` | below 0.25 microsecond | below 0.25 microsecond | below 0.25 microsecond |
| `arc_length` | below 10 microseconds | below 35 microseconds | below 60 microseconds |
| `parameter_at_length` | below 25 microseconds | below 120 microseconds | below 250 microseconds |
| `point_at_length` | below 60 microseconds | below 1,250 microseconds | below 5,000 microseconds |

These are regression targets, not permission to weaken Section 11. Report median, p95, p99, worst non-adaptive time, adaptive count, and adaptive time over at least 10,000 deterministic queries. Separate lookup, elementary evaluation, inverse, and NURBS point costs. Do not quote a best-of-run value alone.

The reference implementation SHOULD use scalar Python/math kernels for small fixed degrees when they benchmark faster than creating NumPy temporaries. `math.fsum`, `math.ulp`, `math.nextafter`, and `math.fma` when available are appropriate fast-path primitives. CPython's `math.atan2` follows the host C library and does not by itself provide a portable correct-rounding proof; its error allowance or certified fallback SHALL be explicit.

# 14. Verification plan

## 14.1 Independent oracle

The acceptance oracle SHALL use at least 256-bit arithmetic and increase to at least 1024 bits for selected cancellation cases. It SHALL independently form $w$, $\sigma$, $\tau$, $G_d$, root counts, the unwrapped phase, and the elementary primitive. Reusing production cell prefixes as expected values is not independent. Numerical quadrature MAY be a supplemental diagnostic but SHALL NOT be the primary oracle.

## 14.2 API and compatibility matrix

Test all four source families:

- `CubicPHSplineOpen`;
- `CubicPHSplineClosed`;
- `PHBSplineOpen`; and
- `PHBSplineClosed`.

For each, verify every public method, scalar type, malformed type, endpoint, four-ULP clamp, out-of-range value, copy, pickle round trip, and source edit isolation. Verify that existing NURBS arrays and point results are unchanged by the addition of metric metadata.

## 14.3 Mandatory geometry cases

The suite SHALL include:

1. `d == 0` on every family;
2. straight spans at several nonzero offsets, for which the offset metric is the source metric;
3. positive and negative cusp-free offsets;
4. a cusp at an interior simple root;
5. an even-multiplicity stationary root at a curvature extremum;
6. a beyond-critical offset with two or more direction reversals;
7. a root at a source endpoint and a root at an internal join;
8. two very close roots and a near-double-root perturbation on each side;
9. high-degree spans at the configured maximum preimage degree;
10. preimages whose continuous phase crosses the principal `atan2` cut and whose tangent turns by more than $2\pi$ on a source span;
11. closed seams with nonzero turning number;
12. large-offset self-intersections;
13. source scales near $10^{-150}$ and $10^{150}$ with independently varied $d$;
14. subnormal local targets near regular endpoints and cusps;
15. cases where $\Delta S$ and $d\Delta\Theta$ cancel by 16, 32, 53, 106, and more than 106 bits; and
16. nonrepresentable total length and forced resource-cap failures.

## 14.4 Required identities and properties

For dense deterministic points, random points, every join, every certified root bracket, and both adjacent floating-point parameters around each root, verify:

- every coefficient identity in Section 4.4, including degree elevation of $\tau$ and the special case $m = 0$;

- equality of the power-basis, interval-restricted Bernstein, and compiled forward/reverse-antiderivative formulas in Sections 4.6.1–4.6.4;
- the casewise `atan2` definition and the winding identity $\Delta\phi = \operatorname{atan2}(Y, X) + 2\pi\omega$ at every ordinary point, cut crossing, cut tangency, and incident cut-root side;
- $0 \le A_d(u) \le L_d$;
- exact-reference strict increase and public nondecrease;
- forward/reverse agreement;
- cell sums equal total length within the certified enclosure;
- the derivative identity
$$A_d'(u) = |1 - d\kappa(u)|\,\|z'(u)\|$$
away from joins and cusps;
- inverse bracketing and the Section 10 acceptance certificate;
- `point_at_length(s)` equals the offset NURBS point at the accepted inverse;
- `cusps` agrees with an independent high-precision root finder in count, parameter (within the representable-boundary limit of Section 3.7), and multiplicity, is empty for zero-offset and straight cases, reports only stationary parameters (the local derivative of `arc_length` collapses there), and survives copy and serialization unchanged;
- zero-offset and straight-offset metric equivalence;
- self-intersection does not change traversal prefixes; and
- construction and queries are deterministic under repeated calls.

For a sign cell, the high-precision oracle SHALL also verify

$$\ell = \eta(\Delta S - d\Delta\Theta).$$

For a whole cusp-free offset whose $1 - d\kappa$ is positive, verify the useful metamorphic identity

$$L_d = L_0 - d\Delta\Theta_{total}.$$

For a sign-reversing offset, verify that each negative signed-speed lobe adds twice its magnitude relative to the signed primitive total.

## 14.5 Inverse sampling

For every case, test at least:

- every stored prefix and its neighboring floats;
- $s/L \in \{0, 2^{-52}, 10^{-12}, 10^{-9}, 0.01, 0.25, 0.5, 0.75, 0.99, 1 - 10^{-9}, 1 - 2^{-52}, 1\}$ when distinct;
- distances immediately before, at, and after every cusp prefix;
- 1,000 deterministic random distances; and
- every ordered-float fallback path through fault injection.

No test may accept only a loose parameter comparison at a cusp. It SHALL check the exact residual or best-representable certificate.

## 14.6 Performance and allocation tests

Benchmark source-span counts from 1 through at least 10,000 and all supported degrees. Verify near-logarithmic lookup scaling, bounded ordinary iteration counts, no ordinary variable-sized buffer or numerical-array allocations, and no construction work on a query. Record adaptive fallback incidence separately. A performance test SHALL fail if an optimization bypasses an accuracy gate.

## 15. Forbidden implementation shortcuts

The following are nonconforming:

- polygonal sampling, chord summation, adaptive numerical quadrature, or fitted approximations as the distance authority;
- generic NURBS arc-length code that ignores the captured PH certificate;
- use of the signed primitive without splitting at every speed-sign change;
- an assumption that all offsets are cusp-free because the source is regular;
- independent principal `atan2` calls without phase unwrapping;
- direct subtraction of large source-length and turning terms without an error bound and cancellation fallback;
- raw `a*b-c*d` classification at a determinant or cusp numerator;
- sampled sign scans or companion roots as proof of complete cusp isolation;
- Newton iteration without a maintained bracket;
- acceptance by parameter-step size alone;
- returning the last iterate after an iteration limit;
- snapping a small cell length, speed, coefficient, root, or residual to zero by a fixed tolerance;
- plain binary64 accumulation of global prefixes;
- rebuilding metric data from public rounded NURBS controls;
- a live back-reference to a mutable source;
- infinite or NaN public distances; and
- a geometry-only handle whose advertised distance methods fail because metric construction was deferred.

## 16. Conformance checklist

An implementation is complete only when all answers are yes.

1. Does every package-produced offset `NURBSHandle` expose the four methods and the `cusps` property in Section 1 with the stated types and validation?
2. Is offset speed evaluated from the verified special identity $|\lambda\sigma^2 - d\tau|/\sigma$?
3. Are all cusp roots and phase crossings completely certified?
4. Is unsigned distance assembled by constant-sign cells?
5. Is tangent phase continuously unwrapped for arbitrary supported degree?
6. Are forward, reverse, local-series, and adaptive-precision evaluators available with explicit error bounds?
7. Are global prefixes stronger than plain binary64 and ordered internally?
8. Is every inverse bracketed, residual checked, and bounded in work?
9. Does the adjacent-float rule handle targets that no representable parameter can match more closely?
10. Are offset construction, serialization, and source-edit isolation atomic?
11. Do high-precision tests cover cusps, reversals, winding, scale, cancellation, and all source families?
12. Do performance reports separate the ordinary path from certified rare fallbacks?
13. Does `cusps` report the complete certified root set of Section 7.3 with multiplicities, in $O(1)$, assembled at construction, and only on handles with a verified metric certificate?

## 17. Primary references


1. R. T. Farouki and T. Sakkalis, "Pythagorean hodograph curves," *IBM Journal of Research and Development* 34(5), 736–752 (1990), https://doi.org/10.1147/rd.345.0736.
2. R. T. Farouki, "Pythagorean-Hodograph Curves: Algebra and Geometry Inseparable," Springer (2008), especially the planar preimage and rational offset construction.
3. R. T. Farouki, "Arc lengths of rational Pythagorean-hodograph curves," *Computer Aided Geometric Design* 34, 1–4 (2015), https://doi.org/10.1016/j.cagd.2015.03.007.
4. G. Albrecht, C. V. Beccari, J.-C. Canonne, L. Romani, "Planar Pythagorean-Hodograph B-Spline curves," *Computer Aided Geometric Design* 57, 57–77 (2017), https://doi.org/10.1016/j.cagd.2017.09.001.

The package's cubic and PH-B-spline technical specifications remain normative for source construction, offset geometry, parameter mapping, exception hierarchy, and existing point evaluation. This addendum supersedes only their statements that `NURBSHandle` has no arc-length operations.